



INSTRUÇÕES

- Esta é uma **atividade individual**, em laboratório, da disciplina “Algoritmos e Estruturas de Dados II”.
- Utilize seu repositório, criado no **GitHub Classroom**, para resolver as questões de programação propostas e enviar suas respostas para correção. **Nenhuma outra forma de entrega da atividade será aceita.**
- Foi disponibilizado um **arquivo .zip no Canvas** com um código-base para esta atividade. Esse arquivo contém:
 - Um “esqueleto” da classe **Pedido** para resolução desta atividade;
 - Classe **App**, com o código implementado na última oficina e novas declarações, assinaturas de métodos e chamadas correspondentes às tarefas desta atividade.
- O prazo **limite para entrega** é o dia **09/03**, às **12h20**. *Commits* no repositório com data posterior a esse limite serão desconsiderados no momento da correção da atividade.
- Além do conteúdo do seu repositório de trabalho e dos materiais disponíveis no Canvas, não é permitida a consulta a nenhum outro material**, porém a professora está **disponível para esclarecer dúvidas** e debater propostas de soluções vindas dos alunos.
- Sendo uma atividade individual, **espera-se dos alunos comprometimento e ética** na produção de sua solução. Soluções produzidas em conjunto, entregas contendo cópias, mesmo que parciais, em código ou estrutura, e consultas a materiais e ferramentas não permitidos são **faltas graves, resultando em nota 0 para todos os envolvidos** e posterior **notificação à Coordenação do Curso de Engenharia de Software** por desrespeito às normas acadêmicas.

Tema: Produtos e Pedidos

Nesta atividade, continuaremos a evolução do nosso sistema. Anteriormente, utilizamos as classes desenvolvidas para carregar e armazenar dados de forma simples em um arquivo de texto. Agora, o objetivo é possibilitar que os produtos sejam vendidos agrupados em Pedidos.

Para acelerar o desenvolvimento, uma versão inicial da classe Pedido foi fornecida. No entanto, o sistema passou por uma evolução nos requisitos, e sua missão será refatorar essa classe e expandir o projeto para suportar as novas regras de negócio.

Pedido
- MAX_PRODUTOS: int = 10 - DESCONTO_PG_A_VISTA: double = 0.15 - produtos: Produto[] - dataPedido: LocalDate - quantProdutos: int - formaDePagamento: int
+ Pedido(dataPedido: LocalDate, formaDePagamento: int) + incluirProduto(novo: Produto) : boolean + valorFinal() : double + toString(): String + equals(obj: Object): boolean

Tarefa 1 (1 ponto): Refatoração da classe Pedido e implementação da classe ItemDePedido

Atualmente, a classe Pedido armazena um vetor de instâncias diretas da classe Produto. Você deverá refatorar a classe Pedido para que, em vez de produtos avulsos, ela armazene instâncias de uma nova classe que representará os itens do carrinho de compras

1.1. Criação da classe ItemDePedido

Crie a classe ItemDePedido, que será responsável por armazenar as informações específicas de cada produto incluído em um pedido. Ela deve conter os seguintes atributos:

- Referência do Produto: Uma referência para o objeto Produto que está sendo vendido.
- Quantidade: A quantidade daquele produto no pedido.

ItemDePedido
- produto: Produto - quantidade: int - precoVenda: double
+ ItemDePedido(produto: Produto, quantidade: int, precoVenda: double) + equals(obj: Object): boolean

- Preço de Venda (precoVenda): O preço unitário do produto no momento exato da transação.
 - O preço de um produto pode ser alterado com o tempo (ex: descontos por proximidade do vencimento). O ItemDePedido deve "congelar" o preço no momento da venda. Se o preço do Produto original sofrer alterações no futuro, o valor registrado no pedido histórico deve permanecer intacto.

1.2. Implementação de Igualdade

Sobrescreva o método *equals* na classe ItemDePedido. A comparação deve retornar true se o Produto atrelado ao item for o mesmo, independentemente da quantidade ou do preço congelado.

Observação: O diagrama apresenta a estrutura básica da classe ItemDePedido. Sinta-se livre para implementar métodos adicionais caso julgue necessário para o bom funcionamento da sua solução.

Tarefa 2 (2 pontos): Lógica de Estoque e Agrupamento de Itens

Em sistemas de vendas, é comum que um cliente inicie um pedido, pause a compra e, posteriormente, crie um novo "carrinho" com mais itens. Antes de finalizar o pagamento, o sistema precisa unificar esses carrinhos pendentes em um único pedido final. Como nossa implementação utiliza arranjos de tamanho fixo, essa operação exige um controle rigoroso dos limites de memória alocada e o correto agrupamento de produtos repetidos.

2.1. O método de Mesclagem

Você deverá implementar um novo método na classe Pedido com a assinatura `public void mesclarPedido(Pedido outroPedido)`. O objetivo é transferir todos os objetos ItemDePedido do pedido secundário para o pedido atual.

- **Atenção à Estrutura (Vetor):** Lembre-se de que os itens do pedido estão armazenados em um **vetor** de tamanho fixo, e não em uma lista dinâmica de tamanho variável.
- **Validação de Capacidade e Exceção:** Antes de realizar qualquer alteração, o método deve analisar se a inserção dos novos itens (considerando apenas os produtos do pedido secundário que ainda não existem no pedido principal) ultrapassará o limite de posições do vetor do pedido atual. Caso a capacidade máxima seja excedida, o método deverá **lançar uma exceção** (você pode utilizar uma exceção padrão do Java, como `IllegalStateException`, ou criar sua própria exceção customizada) e a mesclagem **não** deverá ser executada (operação atômica).
- **Regra de Agrupamento:** Garantido que há espaço no vetor, o sistema deve iterar sobre o vetor do outroPedido. Para cada item válido, utilize o método *equals* (implementado na Tarefa 1) para verificar se aquele produto já existe no vetor do pedido principal.
 - Se o produto **já existir**, o sistema não deve ocupar uma nova posição no vetor. Deve-se apenas **somar a quantidade** do item secundário à quantidade do item principal correspondente.
 - Se o produto **não existir**, o ItemDePedido deve ser inserido na próxima posição livre do vetor principal.
- Após a mesclagem ser concluída com sucesso, o vetor do pedido secundário deve ser logicamente esvaziado (suas posições preenchidas devem ser "limpas" e o contador de itens zerado).

2.2. Regra de Negócio: Resolução de Conflito de Preços

Como os pedidos podem ter sido criados em momentos diferentes, o mesmo produto pode apresentar valores de precoVenda distintos nos dois vetores (por exemplo, o fim de uma promoção relâmpago).

- Durante a mesclagem de itens iguais, o sistema deve assumir como padrão o **menor preço de venda** entre os dois itens, garantindo a vantagem ao cliente e atualizando esse valor no item unificado dentro do vetor principal.

2.3. Geração do Recibo de Venda (imprimirRecibo) Na classe Pedido, crie um método que simule a impressão de um cupom fiscal no terminal. O método deve percorrer o vetor do pedido (tomando o cuidado de ignorar posições vazias/nulas) e exibir:

- Uma listagem organizada contendo: Nome do Produto, Quantidade, Preço Unitário e o Subtotal do item.
- O valor **Total Geral** do pedido.
- *Opcional*: Se a quantidade de um item específico no vetor for superior a 10 unidades, aplique um desconto automático de 5% no subtotal daquele item no momento da impressão.

Outras observações:

- O projeto deve estar hospedado na tarefa correspondente do GitHub Classroom. Endereço para aceitar a tarefa: <https://classroom.github.com/a/lK6JKUsx>
- As atividades pontuadas da disciplina podem depender direta ou indiretamente dos códigos desenvolvidos nas aulas, portanto, é essencial o comprometimento no acompanhamento das atividades semanais.
- Para a correção das atividades pontuadas, serão considerados todo os *commits/pushes* realizados ao longo das semanas, não somente o último com a resposta final do exercício.